

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Facultad de Ingeniería

ACTIVIDAD 2 — DE JSON A MYSQL

Tienda virtual de ropa «Manos del Oriente»

Migración del catálogo a MySQL con PDO, repositorios, CRUD de prendas y socios, paginación y resistencia a la inyección SQL

Integrantes:

1. Daniela Cayo Mamani
2. Nombre completo (GitHub: RonSalvet)
3. Nombre completo (GitHub: tolavi)
4. Nombre completo (GitHub: JUANJOVICO2021)
5. Nombre completo (GitHub: leinn04)

Carrera: Ingeniería de Sistemas

Asignatura: Tecnología Web II

Docente: M.Sc. Ing. Jimmy Nataniel Requena Llorentty

Santa Cruz de la Sierra, Bolivia

13 de octubre de 2026

Índice

Resumen 4
Introducción 5
Marco teórico 6
Modelo relacional y normalización 6
Integridad referencial y transacciones 6
PDO y consultas preparadas 6
Inyección SQL 7
Patrón repositorio 7
Metodología 8
Resultados 9
Paso 1. Alcance de la actividad 9
Paso 2. Requisitos de la actividad 10
Paso 3. Modelo entidad-relación y script SQL 11
Paso 4. Conexión PDO fuera del código público 14
Paso 5. Repositorios, CRUD y paginación 14
Paso 6. Resistencia a la inyección SQL 19
Paso 7. Migración de los datos de la Actividad 1 21
Paso 8. Pruebas automáticas 21
Discusión 24
Conclusiones 26
Referencias 27
Anexos 29
Anexo A. Registro de uso de inteligencia artificial 29
Anexo B. Matriz de trazabilidad de la Actividad 2 30
Anexo C. Glosario 30

Índice de figuras y tablas

Tabla 1. Alcance de la Actividad 2: dentro y fuera 9
Tabla 2. Backlog de la Actividad 2 en el tablero de GitHub Projects 10
Tabla 3. Requisitos agregados en la Actividad 2 11
Figura 1. Modelo entidad-relación de la base de datos manos_del_oriente 12
Tabla 4. Decisiones de diseño de la base de datos 13
Tabla 5. Verificación manual de los datos de ejemplo 13
Figura 2. Creación de la conexión PDO en la clase Conexion 14
Figura 3. Paginación con parámetros enlazados en ProductoRepositorio::paginar() 15
Figura 4. Tienda paginada de ocho en ocho con los datos de MySQL 16
Figura 5. Gestión de prendas con el resumen del inventario calculado por MySQL 17
Figura 6. Módulo de socios con paginación de cinco en cinco 18
Figura 7. Formulario de socio con errores de validación por campo 19
Tabla 6. Ataques de inyección SQL probados y resultado obtenido 20
Figura 8. Resultado de buscar 20
Tabla 7. Resumen de las comprobaciones automáticas 21
Figura 9. Ejecución de las pruebas automáticas en el entorno Docker del curso 22
Tabla 8. Fallos de prueba originados en propuestas de la IA y su causa real 23
Tabla 9. Registro de uso de IA en la Actividad 2 29
Tabla 10. Matriz de trazabilidad: requisito, issue, implementación y prueba 30

Resumen

Este informe presenta la segunda actividad del proyecto «Manos del Oriente», el sistema web de una asociación de confeccionistas textiles de la Chiquitanía que se desarrolla de forma incremental en la asignatura Tecnología Web II. En la primera actividad el catálogo guardaba las prendas en archivos JSON; en esta se migró la persistencia a una base de datos MySQL. Se diseñó un modelo entidad-relación de cinco tablas con cuatro claves foráneas, restricciones UNIQUE y CHECK; se configuró la conexión PDO fuera del código público y sin emulación de consultas preparadas; y se implementaron dos repositorios, ProductoRepositorio y SocioRepositorio, que concentran el CRUD con consultas preparadas, transacciones y paginación. Se agregó un CRUD web de socios, se migraron los datos de la primera actividad y se demostró con trece ataques que la aplicación resiste la inyección SQL. La verificación se realizó con 57 comprobaciones automáticas ejecutadas con cero fallos tanto en Windows (PHP 8.4 y MariaDB 10.4) como en el entorno Docker del curso (PHP 8.3 y MySQL 8.4). El trabajo se organizó en un tablero de sprint de GitHub Projects y se registró el uso de inteligencia artificial: de las propuestas de la IA, cinco contenían errores que las propias pruebas pusieron en evidencia y que el equipo corrigió.

Palabras clave: PHP 8, MySQL, PDO, consultas preparadas, inyección SQL, patrón repositorio, paginación, modelo entidad-relación

Introducción

Un catálogo guardado en archivos JSON funciona mientras lo usa una sola persona y los datos son pocos, pero no garantiza que dos ventas simultáneas no se pisen, que una prenda no quede asociada a una socia inexistente ni que el inventario se pueda consultar con criterios combinados. Esas garantías son precisamente las que ofrece un sistema gestor de bases de datos relacional (Elmasri y Navathe, 2016), y son las que necesita la Asociación de Confeccionistas Textiles «Manos del Oriente» para que su tienda virtual pueda crecer hacia pedidos, usuarios y reportes en las siguientes actividades.

El objetivo de esta actividad fue migrar la aplicación de la Actividad 1 de JSON a MySQL sin perder ninguna de sus funciones. Los objetivos específicos fueron: (a) diseñar el modelo entidad-relación y el script SQL con tablas, claves foráneas y datos de ejemplo; (b) configurar la conexión PDO fuera del código público; (c) implementar ProductoRepositorio y SocioRepositorio con CRUD, validación y paginación; (d) demostrar que la aplicación resiste la inyección SQL; y (e) registrar el uso de inteligencia artificial en la bitácora del proyecto.

El documento se organiza como sigue: el marco teórico reúne los conceptos que sostienen las decisiones técnicas; la metodología describe cómo se organizó y verificó el trabajo; los resultados presentan, paso a paso, el alcance, el modelo de datos, la conexión, los repositorios, la defensa contra la inyección SQL, la migración y las pruebas; la discusión interpreta los hallazgos y sus limitaciones; y las conclusiones cierran con el trabajo futuro. Los anexos contienen el registro de uso de inteligencia artificial, la matriz de trazabilidad y un glosario.

Marco teórico

Modelo relacional y normalización

Codd (1970) propuso representar los datos como relaciones, es decir, tablas de filas sin orden definido, de modo que las aplicaciones no dependan de cómo se almacenan físicamente. Elmasri y Navathe (2016) explican que la normalización organiza esas tablas para evitar redundancia y anomalías de actualización; la primera forma normal exige que cada celda contenga un único valor atómico. Ese principio justifica que las tallas de una prenda se guarden en una tabla intermedia y no como un texto «S,M,L» dentro de la tabla de productos.

Integridad referencial y transacciones

Una clave foránea obliga a que el valor de una columna exista en la tabla referenciada, y sus acciones ON DELETE definen qué ocurre al borrar el registro padre (Oracle Corporation, s.f.). Härder y Reuter (1983) formularon las propiedades ACID de una transacción: atomicidad, consistencia, aislamiento y durabilidad. En este proyecto la atomicidad es la que importa: guardar una prenda implica insertar una fila en productos y varias en producto_tallas, y si una de ellas falla no debe quedar nada a medias.

PDO y consultas preparadas

PHP Data Objects (PDO) es la capa de acceso a datos de PHP que ofrece una interfaz común para distintos gestores (The PHP Group, s.f.-a). Una consulta preparada separa el texto SQL de los datos: el servidor compila primero la consulta con marcadores y después recibe los valores, por lo que un dato nunca se interpreta como código (The PHP Group, s.f.-b). Por defecto, el controlador de MySQL emula las consultas preparadas armando el texto en PHP; desactivar la emulación hace que la separación ocurra en el propio servidor.

Inyección SQL

La inyección SQL ocurre cuando un dato del usuario se concatena dentro de una consulta y altera su estructura. Halfond et al. (2006) clasifican sus variantes —tautologías como ' OR '1'='1, consultas UNION y consultas encadenadas, entre otras—, y la categoría de inyección figura entre los riesgos más críticos de las aplicaciones web (OWASP Foundation, 2021). La defensa principal recomendada es usar consultas parametrizadas y, para las partes de la consulta que no admiten parámetros, como ORDER BY, una lista blanca de valores permitidos (OWASP Foundation, s.f.).

Patrón repositorio

Fowler (2002) describe el repositorio como un objeto que media entre el dominio y la capa de datos y que se comporta como una colección de objetos en memoria. Las páginas de la aplicación piden prendas o socios al repositorio sin conocer el SQL, y todo el acceso a la base queda concentrado en una sola clase por entidad, donde es más fácil revisarlo y probarlo.

Metodología

El desarrollo siguió un enfoque incremental sobre la base de la Actividad 1: primero se diseñó el modelo de datos, después la conexión, luego los repositorios y, por último, se adaptaron las páginas, que dejaron de usar la clase Catalogo para usar los repositorios. Cada unidad de trabajo se registró como un issue en el tablero de sprint del proyecto en GitHub Projects (https://github.com/orgs/TecnologiaWeb2/projects/1), con las columnas Backlog, To Do, In Progress, In Review y Done, y con un hito (milestone) por actividad; el código se versionó en el repositorio https://github.com/TecnologiaWeb2/Tecnologiaweb2 y cada actividad entregada se marca con una etiqueta (actividad-1, actividad-2).

Se utilizó Claude Code (Anthropic, 2026) como asistente de programación con acceso al repositorio. La regla de trabajo fue que la IA propone y el equipo decide: ninguna propuesta se aceptó sin ejecutarla y contrastarla con la base de datos o con un cálculo manual. Por esa razón las pruebas automáticas se escribieron junto con el código y no al final, siguiendo la idea de Myers et al. (2011) de que probar es ejecutar el programa con la intención de encontrar defectos.

Las pruebas se ejecutaron en dos entornos para comprobar la portabilidad: Windows 11 con PHP 8.4.13 y MariaDB 10.4.32 de XAMPP, y el entorno Docker entregado por el docente (Ubuntu 24.04 con PHP 8.3 y MySQL 8.4). Las pruebas usan una base de datos propia (manos_del_oriente_pruebas), que se crea y se borra en cada ejecución, y una carpeta temporal para las fotos, de modo que nunca modifican los datos reales.

Resultados

Paso 1. Alcance de la actividad

La Tabla 1 delimita qué se comprometió en esta actividad y qué se dejó para las siguientes. El criterio fue que la aplicación conservara todas las funciones de la Actividad 1, ahora sobre MySQL, y que se agregara la gestión de socios, que en el JSON era solo un texto libre.

Tabla 1

Alcance de la Actividad 2: dentro y fuera

Dentro de la actividad	Fuera de la actividad
Modelo entidad-relación y script SQL con datos de ejemplo.	Usuarios, inicio de sesión y roles (Actividad 3).
Conexión PDO configurada fuera de public/ y de git.	Pedidos y ventas con descuento de stock (Actividad 3).
CRUD de prendas y de socios con validación.	Reportes de ventas e inventario (Actividad 3).
Paginación de la tienda, la gestión y los socios.	API REST y consumo asíncrono con fetch (Actividad 4).
Demostración de resistencia a la inyección SQL.	Publicación en un hosting (proyecto final).
Migración de los datos JSON de la Actividad 1.	Integración continua en GitHub Actions.

Nota. Elaboración propia. Lo que está fuera no es lo que el sistema no podría hacer, sino lo que se decidió no comprometer en esta actividad; cada elemento figura como issue en la columna Backlog del tablero.

El trabajo se organizó en el tablero de sprint. La Tabla 2 presenta los issues de la Actividad 2 con el requisito que atienden y su estado al cierre de este informe.

Tabla 2

Backlog de la Actividad 2 en el tablero de GitHub Projects

Issue	Unidad de trabajo	Requisito	Estado
#9	Modelo entidad-relación	RNF-08	Done
#10	Script SQL: esquema y datos de demostración	RNF-08	Done
#11	Conexión PDO configurada fuera de public/	RNF-07	Done
#12	ProductoRepositorio: CRUD, transacciones y paginación	RF-07, RNF-09	Done
#13	SocioRepositorio y CRUD web de socios	RF-06	Done
#14	Demostración de resistencia a inyección SQL	RNF-06	Done
#15	Migración de los datos JSON a MySQL	RF-09	Done
#16	Pruebas automáticas de la Actividad 2 (57/57)	Todos	Done
#17	Capturas de pantalla de la versión con MySQL	—	In Progress
#18	Informe APA y presentación de la Actividad 2	—	In Progress
#19	Completar la bitácora de IA e integrantes	—	To Do
#20	Etiqueta actividad-2 y entrega	—	To Do

Nota. Elaboración propia a partir del tablero https://github.com/orgs/TecnologiaWeb2/projects/1. Los requisitos se definen en la Tabla 3.

Paso 2. Requisitos de la actividad

A los requisitos de la Actividad 1 (RF-01 a RF-05 y RNF-01 a RNF-05), que se mantienen, se agregaron los de la Tabla 3. Cada uno tiene al menos una prueba automática que lo verifica (Anexo B).

Tabla 3

Requisitos agregados en la Actividad 2

Código	Requisito	Tipo
RF-06	Registrar, editar, buscar y eliminar socios; no se puede eliminar un socio con prendas	Funcional
RF-07	Paginar la tienda (8 por página), la gestión (10) y los socios (5)	Funcional
RF-08	Mostrar en la ficha de cada prenda la socia que la confeccionó	Funcional
RF-09	Migrar las prendas del JSON de la Actividad 1 a MySQL	Funcional
RNF-06	Resistir la inyección SQL en toda entrada del usuario	Seguridad
RNF-07	Guardar la configuración de la conexión fuera de public/ y del repositorio	Seguridad
RNF-08	Garantizar la integridad en la base: claves foráneas, UNIQUE y CHECK	Fiabilidad
RNF-09	Guardar cada prenda y sus tallas en una sola transacción	Fiabilidad
RNF-10	Ante un error de la base, responder 503 sin revelar detalles técnicos	Seguridad

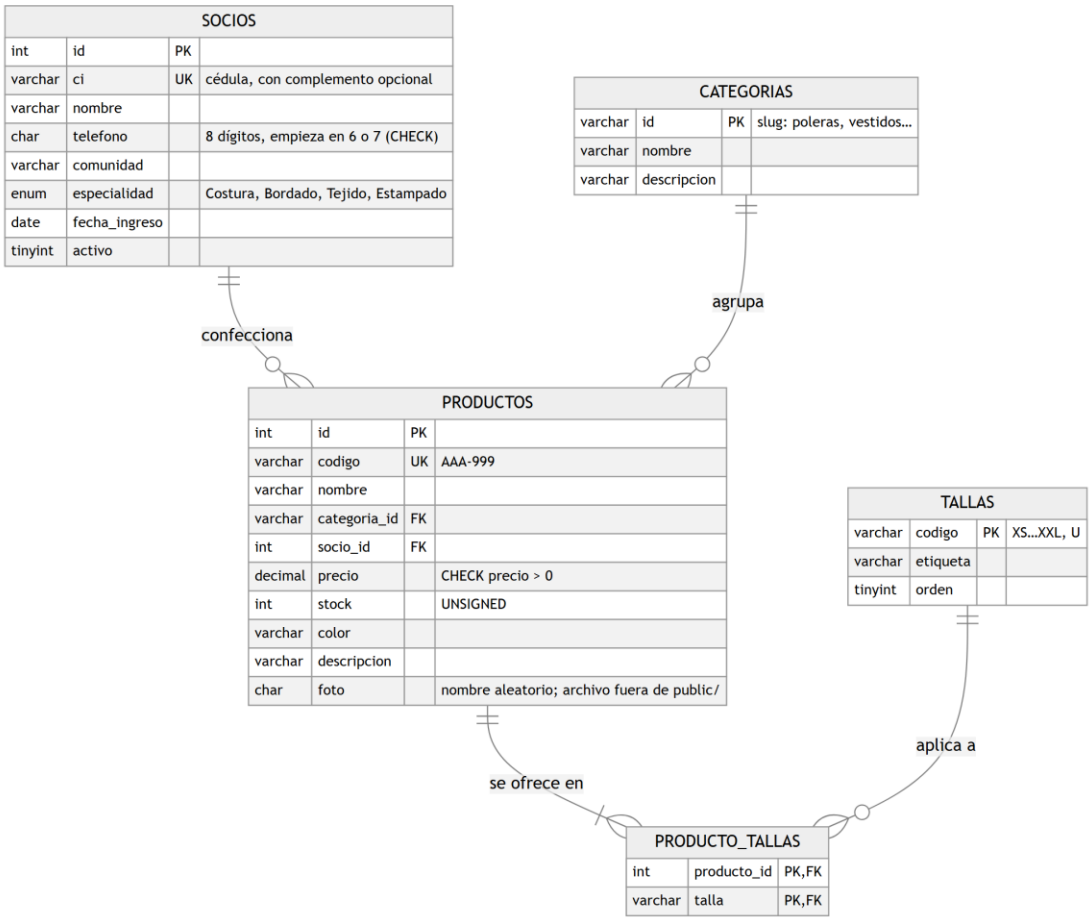
Nota. RF = requerimiento funcional; RNF = requerimiento no funcional. La numeración continúa la de la Actividad 1.

Paso 3. Modelo entidad-relación y script SQL

El modelo (Figura 1) tiene cinco tablas. La decisión más importante fue convertir al socio en una entidad propia: en el JSON de la Actividad 1 la persona que confeccionaba la prenda era un texto libre, lo que permitía escribir el mismo nombre de varias formas y no permitía saber cuántas prendas aporta cada socia. La relación entre prendas y tallas es de muchos a muchos y se resolvió con la tabla intermedia producto_tallas, conforme a la primera forma normal (Elmasri y Navathe, 2016).

Figura 1

Modelo entidad-relación de la base de datos manos_del_oriente



Nota. Elaboración propia con apoyo de IA, escrito como código Mermaid en docs/actividad-02/MODELO_ER.md. PK = clave primaria; FK = clave foránea; UK = clave única.

Las reglas del negocio no viven solo en PHP: también se declararon en la base, de modo que se cumplan aunque alguien inserte datos por otra vía. La Tabla 4 resume esas decisiones y su justificación.

Tabla 4

Decisiones de diseño de la base de datos

Decisión	Motivo
producto_tallas (muchos a muchos) en lugar de una columna «S,M,L»	Primera forma normal: cada talla es una fila, se puede filtrar con EXISTS y la clave foránea impide tallas inexistentes.
ON DELETE RESTRICT en productos.socio_id y productos.categoria_id	No se puede borrar una socia ni una categoría que tiene prendas: no quedan prendas huérfanas.
ON DELETE CASCADE en producto_tallas.producto_id	Al borrar una prenda, sus tallas se borran solas.
UNIQUE en productos.codigo y socios.ci	Última defensa contra duplicados aunque dos personas guarden al mismo tiempo.
CHECK en precio y teléfono, UNSIGNED en stock, ENUM en especialidad	El precio debe ser mayor que cero y el celular boliviano tiene 8 dígitos y empieza en 6 o 7.
DECIMAL(10,2) para el precio	Los montos de dinero no se guardan en coma flotante.
utf8mb4 e InnoDB	Acentos y eñes (Chuvé, saó) y soporte de claves foráneas y transacciones.

Nota. Elaboración propia a partir de bd/esquema.sql.

El script bd/datos.sql carga 6 socios (uno inactivo), 5 categorías, 7 tallas y 14 prendas, y bd/instalar.php crea la base y ejecuta ambos scripts. Los datos de ejemplo se verificaron a mano antes de usarlos como valores esperados en las pruebas (Tabla 5).

Tabla 5

Verificación manual de los datos de ejemplo

Socio	Comunidad	Prendas	Estado
Rosa Chuvé Supepí	Concepción	3	Activo
Elena Masabí Tomichá	San Javier	3	Activo
Mario Surubí Peña	San Ignacio de Velasco	3	Activo
Juana Pesoa Rojas	Concepción	3	Activo
Lidia Choré Vaca	San Rafael	2	Activo
Carmen Parapaino Masaí	Santa Ana de Velasco	0	Inactivo
Total		**14**	

Nota. El inventario suma 157 unidades y un valor de Bs 19.538,50 (Σ precio × stock). El mismo valor se obtuvo con SUM(precio * stock) en MySQL y aparece en la página de gestión (Figura 4).

Paso 4. Conexión PDO fuera del código público

Los datos de conexión viven en config/config.php, que está fuera de public/ —la única carpeta que expone el servidor— y figura en .gitignore, por lo que nunca se sube al repositorio; en su lugar se versiona la plantilla config/config.ejemplo.php. Cada valor puede reemplazarse con variables de entorno (TIENDA_BD_… o las BD_… del Docker del curso), lo que permite usar otra base en las pruebas y otras credenciales en el hosting sin tocar el código. La clase Conexion (Figura 2) crea una sola conexión por petición con dos atributos clave: los errores lanzan excepciones y la emulación de consultas preparadas está desactivada.

Figura 2

Creación de la conexión PDO en la clase Conexion

```
public static function crear(array $bd, bool $conBase = true): PDO
{
    $dsn = "mysql:host={$bd['host']};port={$bd['puerto']};charset=utf8mb4"
         . ($conBase ? ";dbname={$bd['nombre']}" : '');
    return new PDO($dsn, $bd['usuario'], $bd['clave'], [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,      // errores como excepciones
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,              // preparadas REALES
    ]);
}
```

Nota. Fragmento de src/Conexion.php. Con la emulación desactivada, MySQL recibe la consulta y los datos por separado (The PHP Group, s.f.-b).

Si la base de datos no responde, el manejador de errores de src/arranque.php registra el detalle en el log del servidor y muestra al visitante una página 503 con un mensaje genérico, sin SQLSTATE, rutas ni credenciales (RNF-10).

Paso 5. Repositorios, CRUD y paginación

ProductoRepositorio y SocioRepositorio reemplazan a la clase Catalogo de la Actividad 1. Como en esa actividad toda la persistencia ya estaba concentrada en una sola clase, la migración no obligó a reescribir los modelos: Producto y Categoria siguen siendo clases readonly, y se agregó Socio con la misma técnica de promoción de propiedades. Los repositorios

ofrecen buscarPorCodigo(), paginar(), crear(), actualizar() y eliminar(), todos con consultas preparadas.

Crear o actualizar una prenda modifica dos tablas, por lo que ambas operaciones se ejecutan dentro de una transacción: si falla la inserción de una talla o la socia no existe, se ejecuta ROLLBACK y no queda nada guardado (RNF-09). La paginación usa LIMIT y OFFSET enlazados como enteros con PDO::PARAM_INT, y el número de página que llega por la URL se normaliza con la clase Pagina, de modo que valores como «abc», «-3» o «999» no producen errores (Figura 3).

Figura 3

Paginación con parámetros enlazados en ProductoRepositorio::paginar()

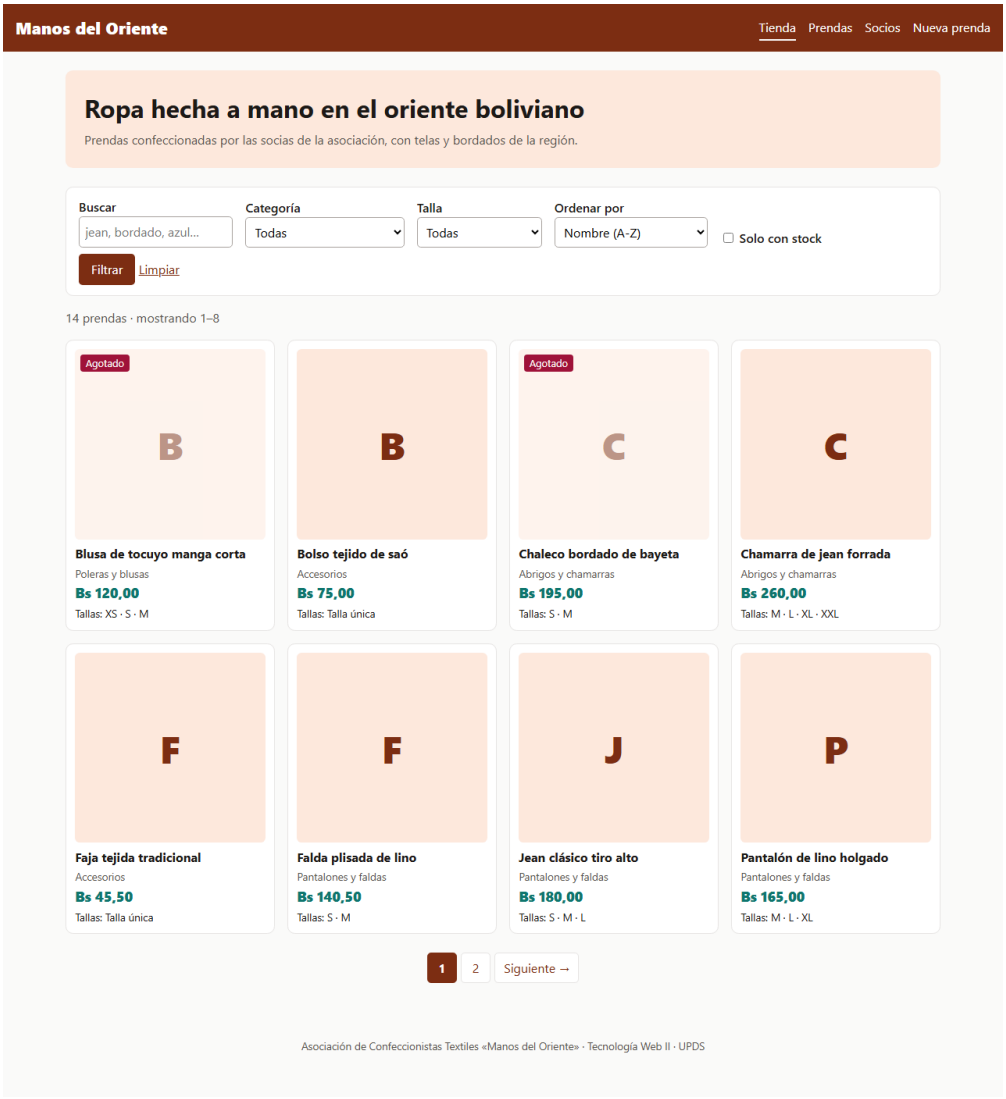
```
$ordenSql = self::SQL_ORDEN[$orden] ?? self::SQL_ORDEN['nombre'];   // lista blanca
$consulta = $this->pdo->prepare("SELECT ... FROM productos p ... $donde
                                 ORDER BY $ordenSql LIMIT :limite OFFSET :desde");
foreach ($parametros as $clave => $valor) {
    $consulta->bindValue($clave, $valor, is_int($valor) ? PDO::PARAM_INT :
PDO::PARAM_STR);
}
$consulta->bindValue('limite', $porPagina, PDO::PARAM_INT);
$consulta->bindValue('desde', ($pagina - 1) * $porPagina, PDO::PARAM_INT);
```

Nota. Fragmento simplificado de src/Repositorio/ProductoRepositorio.php. $donde solo contiene condiciones fijas con marcadores (:t1, :categoria…); ningún dato del usuario se concatena en el SQL.

Las páginas se adaptaron sin cambiar su comportamiento visible. La tienda muestra ocho prendas por página con sus filtros GET (Figura 4), la gestión muestra diez por página con el resumen del inventario calculado por MySQL (Figura 5) y el nuevo módulo de socios pagina de cinco en cinco, indica cuántas prendas aportó cada socio y desactiva el botón de eliminar cuando tiene prendas (Figura 6).

Figura 4

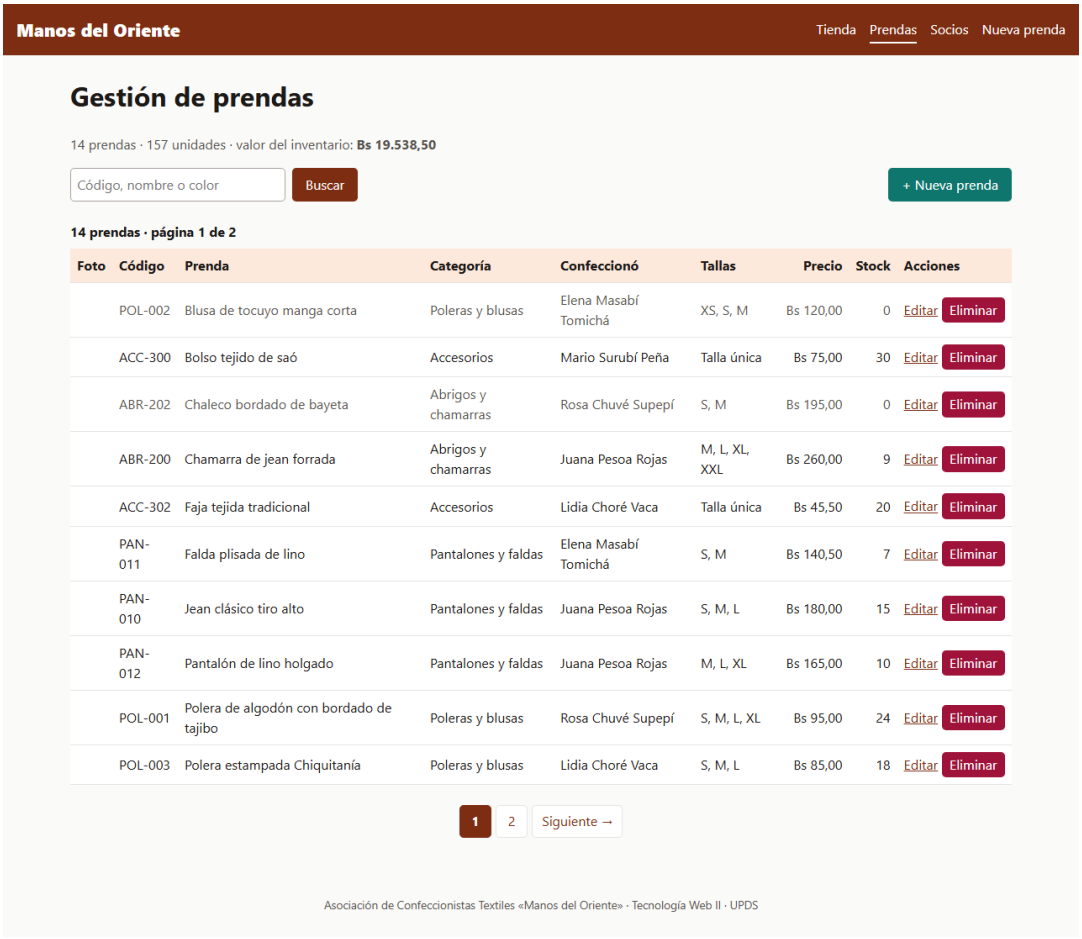
Tienda paginada de ocho en ocho con los datos de MySQL



Nota. Captura del entorno Docker del curso (http://localhost:8005). Las 14 prendas se reparten en dos páginas; las agotadas llevan la insignia «Agotado».

Figura 5

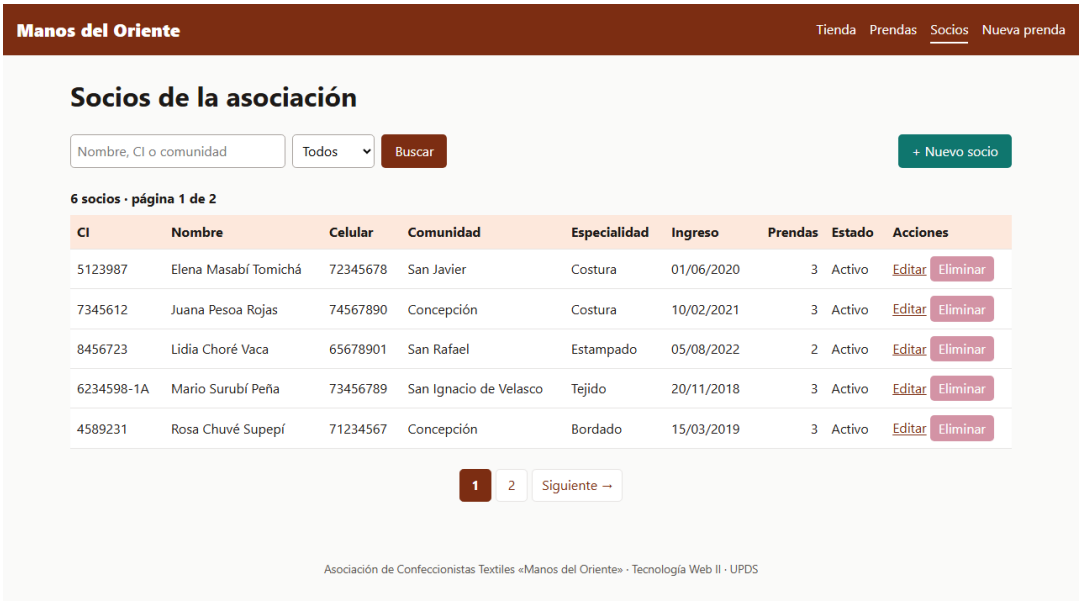
Gestión de prendas con el resumen del inventario calculado por MySQL



Nota. El resumen (14 prendas, 157 unidades, Bs 19.538,50) coincide con la verificación manual de la Tabla 5. La columna «Confeccionó» proviene de la relación con la tabla socios.

Figura 6

Módulo de socios con paginación de cinco en cinco

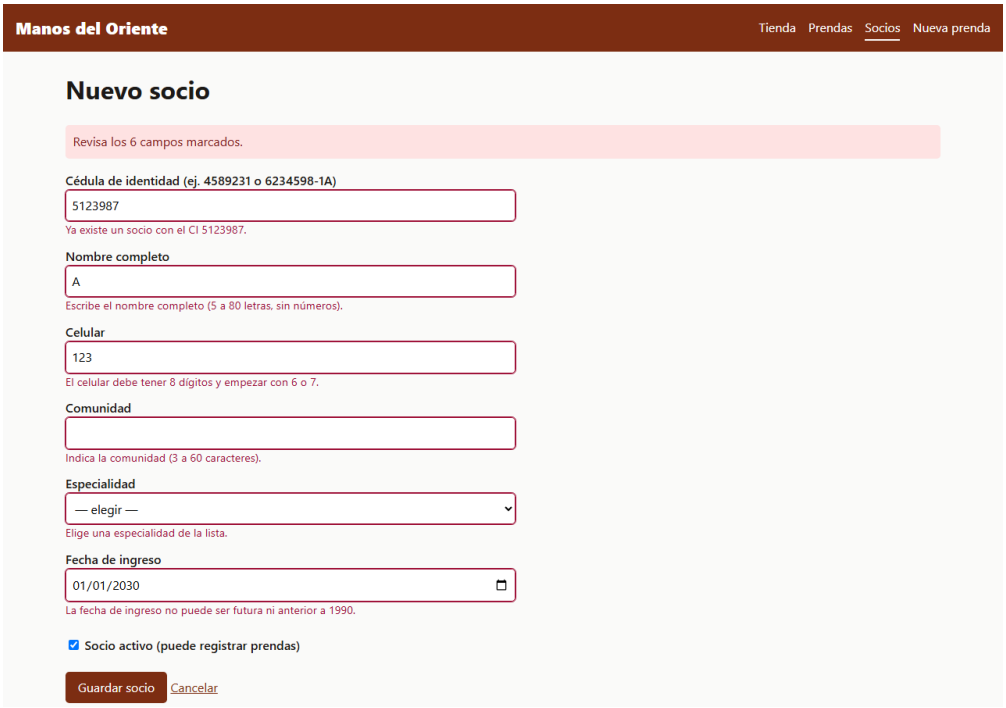


Nota. El botón «Eliminar» aparece desactivado porque los cinco socios tienen prendas; la clave foránea con ON DELETE RESTRICT lo impediría también desde la base.

Los formularios de prendas y de socios conservan la validación de la Actividad 1 y agregan las reglas nuevas: cédula de identidad con complemento opcional, celular boliviano, fecha de ingreso no futura y CI no repetido. Ante datos inválidos la página responde con el código 422 y muestra el mensaje junto a cada campo (Figura 7).

Figura 7

Formulario de socio con errores de validación por campo



Nota. Respuesta 422 a un envío con CI repetido, nombre corto, celular inválido, comunidad y especialidad vacías y fecha de ingreso futura.

Paso 6. Resistencia a la inyección SQL

La defensa combina cinco medidas: consultas preparadas reales para todos los datos; una lista blanca para ORDER BY, que no admite parámetros; el escape de los comodines % y _ en las búsquedas LIKE; la validación de enteros con filter_var() y su enlace con PDO::PARAM_INT; y mensajes de error sin detalles técnicos (OWASP Foundation, s.f.). Para demostrarlo se probaron trece ataques, de los cuales la Tabla 6 presenta los más representativos, y una prueba adicional revisa el código fuente de los repositorios y falla si algún prepare(), query() o exec() interpola una variable que no pertenezca a las listas blancas.

Tabla 6

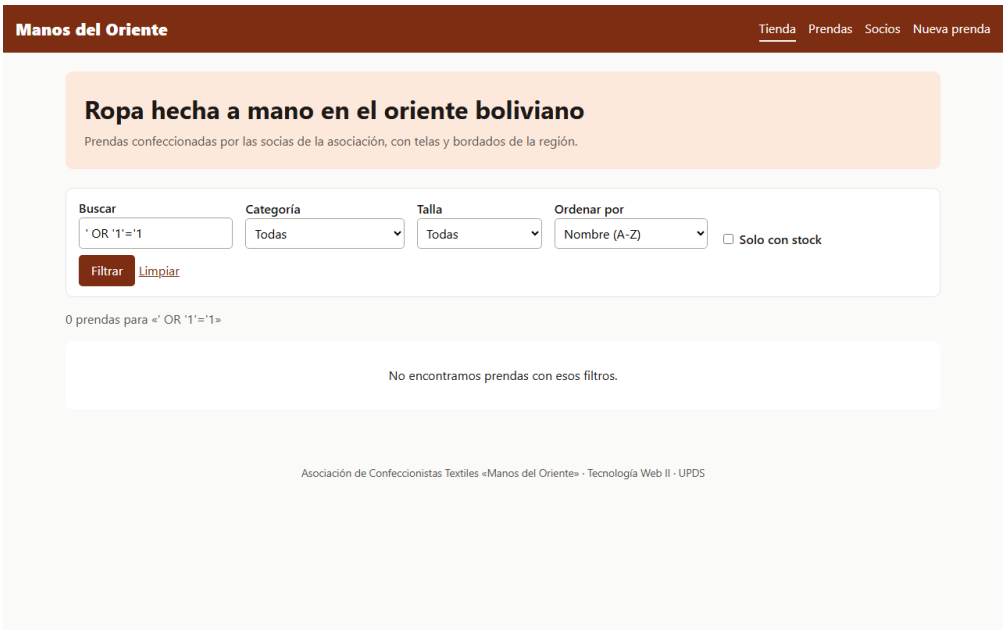
Ataques de inyección SQL probados y resultado obtenido

Ataque	Dónde	Resultado
' OR '1'='1	Búsqueda de la tienda (/?q=)	0 prendas, sin error SQL
POL-001' OR '1'='1	Ficha (/producto.php?codigo=)	404: no muestra otra prenda
precio; DROP TABLE productos; --	Orden (/?orden=)	Se ignora por la lista blanca; tablas intactas
1 OR 1=1	Paginación (/admin.php?pagina=)	Se muestra la página 1 normal
' UNION SELECT 1,2,3,... --	Búsqueda de socios (/socios.php?q=)	0 socios
Polera'); DROP TABLE socios;-- <script>	Nombre en el formulario de alta	Se guarda literal y se muestra escapado
% y _	ProductoRepositorio::paginar(texto:)	Se buscan literalmente: 0 prendas
' OR ''='	SocioRepositorio::existeCi()	Devuelve false

Nota. Elaboración propia. Todos los ataques forman parte de las pruebas automáticas de verificar.php; el detalle y una guía para repetirlos en vivo están en docs/actividad-02/INYECCION_SQL.md.

Figura 8

Resultado de buscar «' OR '1'='1» en la tienda



Nota. La tautología se busca como texto literal: ninguna prenda contiene esa cadena y la página responde normalmente, sin mensajes de error de la base.

Paso 7. Migración de los datos de la Actividad 1

El script bd/migrar_json.php lee datos/productos.json de la Actividad 1 y lo inserta en MySQL dentro de una sola transacción, de modo que la migración se aplica completa o no se aplica. Las prendas cuyo código ya existe se omiten, y el modo --simular informa qué haría sin escribir nada. En la simulación sobre los datos de demostración, siete prendas se omitieron porque ya existían y una se migró como nueva.

Paso 8. Pruebas automáticas

El archivo verificar.php, con el motor Verificador.php de la asignatura, reúne 57 comprobaciones. La primera parte prueba la base y los repositorios directamente; la segunda levanta el servidor embebido de PHP y envía peticiones HTTP reales con cURL, incluidos los ataques de inyección (Tabla 7).

Tabla 7

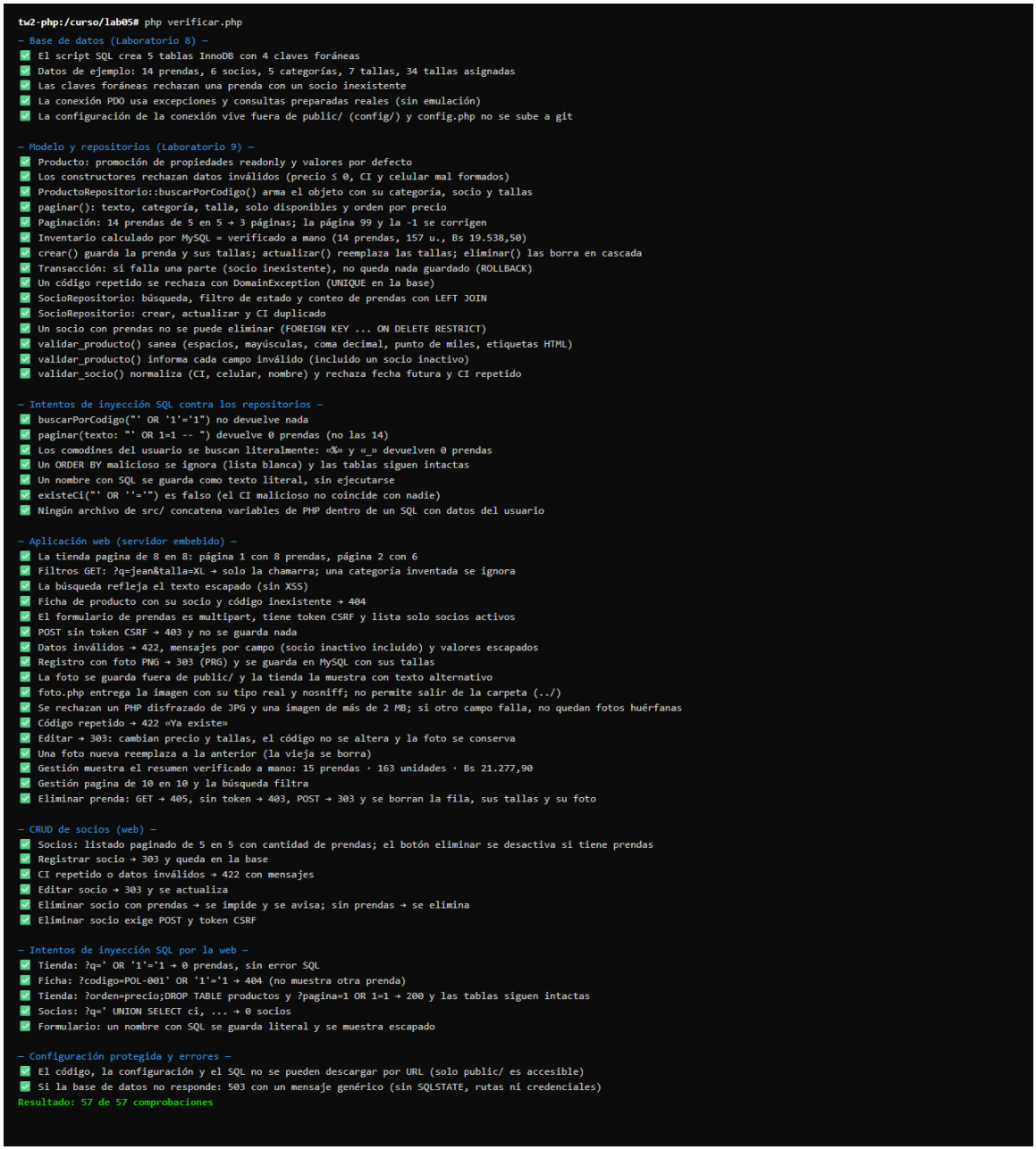
Resumen de las comprobaciones automáticas

Grupo	Qué se comprueba	N.º
Base de datos	Cinco tablas InnoDB con cuatro claves foráneas, datos de ejemplo, rechazo de una FK inválida, atributos de PDO y configuración protegida	5
Modelo y repositorios	Promoción de propiedades, reglas de los constructores, búsqueda, filtros, paginación, inventario verificado a mano, CRUD, transacción con ROLLBACK, UNIQUE, RESTRICT y validación	15
Inyección contra los repositorios	Tautologías, comentarios, comodines, ORDER BY malicioso, SQL guardado como texto y revisión automática del código fuente	7
Aplicación web	Paginación, filtros, escape, ficha y 404, CSRF, 422, PRG, fotos seguras, edición, eliminación y resumen de la gestión	17
CRUD de socios	Listado paginado, alta, CI repetido, edición y eliminación con y sin prendas	6
Inyección por la web	Ataques por la URL y por el formulario	5
Configuración y errores	El código y la configuración no se descargan por URL; error 503 genérico	2
Total		**57**

Nota. Elaboración propia a partir de la salida de php verificar.php.

Figura 9

Ejecución de las pruebas automáticas en el entorno Docker del curso



Nota. Salida de php verificar.php dentro del contenedor tw2-php (PHP 8.3 y MySQL 8.4): 57 de 57 comprobaciones satisfactorias. El mismo resultado se obtuvo en Windows con PHP 8.4 y MariaDB 10.4.

Durante el desarrollo, varias pruebas fallaron por errores de la propia IA al escribirlas, no de la aplicación. La Tabla 8 los documenta, porque cada uno obligó a entender la causa antes de corregir.

Tabla 8

Fallos de prueba originados en propuestas de la IA y su causa real

Fallo observado	Causa real	Corrección
Se esperaban 5 claves foráneas	El esquema tiene 4; la IA contó mal	Se corrigió el valor esperado
Se esperaba que la prenda nueva tuviera el id 15	Un INSERT fallido por la FK ya había consumido el autoincremento	La prueba busca la prenda por su código y no por el id
Una prueba con «saó» en la URL fallaba	La prueba enviaba la URL sin codificar	Se codificó la URL en la prueba
Se esperaba 404 en /../config/config.php	El servidor embebido devuelve la página de inicio	Se comprueba que no se filtra el contenido
56 de 57 en Docker	PHP 8.3 devuelve 0 y PHP 8.4 devuelve false para ATTR_EMULATE_PREPARES	La prueba acepta ambos valores

Nota. Elaboración propia a partir de docs/BITACORA_IA.md. En ningún caso el error estaba en la aplicación.

Discusión

El hallazgo técnico más relevante es que la decisión de la Actividad 1 de concentrar la persistencia en una sola clase redujo la migración a reemplazar esa clase por dos repositorios. Las páginas, los formularios y las plantillas cambiaron muy poco, y las pruebas de la Actividad 1 que seguían siendo válidas pasaron sin modificarse. Es la ventaja que Fowler (2002) atribuye al patrón repositorio: el resto de la aplicación no depende de cómo se guardan los datos.

El segundo hallazgo es que la seguridad frente a la inyección SQL no depende de una sola medida. Las consultas preparadas resuelven la mayoría de los casos, pero no protegen ORDER BY ni los comodines de LIKE: un usuario que escribe «%» obtendría todo el catálogo aunque la consulta esté parametrizada. Por eso se agregaron la lista blanca, el escape de comodines y una prueba que revisa el código fuente, de modo que una concatenación futura se detecte antes de llegar a producción.

Sobre el uso de inteligencia artificial, el patrón fue el mismo de la Actividad 1: la IA produjo con rapidez el esquema, los repositorios y las pruebas, pero cinco de sus valores o supuestos eran incorrectos (Tabla 8). Ninguno se habría detectado sin ejecutar las pruebas en dos entornos distintos; el último apareció solo al pasar de PHP 8.4 a PHP 8.3. La lección es que una prueba que falla puede estar mal escrita, y que hay que entender la causa antes de «arreglar» el código.

Se reconocen tres limitaciones. Primero, quienes escribieron el código son quienes escribieron las pruebas, sin una revisión independiente. Segundo, la aplicación todavía no tiene usuarios ni roles, de modo que cualquiera que acceda a la gestión puede modificar el catálogo; esto se resuelve en la Actividad 3. Tercero, la paginación con OFFSET es adecuada para cientos

de prendas, pero se vuelve lenta con tablas muy grandes; para este caso no es un problema, pero conviene saberlo.

Conclusiones

Se migró la tienda virtual «Manos del Oriente» de JSON a MySQL cumpliendo todos los requisitos de la actividad: un modelo entidad-relación de cinco tablas con integridad declarada en la base, una conexión PDO configurada fuera del código público y sin emulación de consultas preparadas, dos repositorios con CRUD, transacciones y paginación, un nuevo módulo de socios, la migración de los datos anteriores y una demostración verificable de resistencia a la inyección SQL. Las 57 comprobaciones automáticas pasaron en los dos entornos de prueba.

Como trabajo futuro, en el orden en que se abordará: incorporar usuarios con contraseñas cifradas, sesiones seguras y roles; registrar pedidos que descuenten el stock dentro de una transacción; generar reportes de ventas e inventario; exponer una API REST consumida con fetch; ejecutar las pruebas en GitHub Actions en cada cambio; y publicar el sistema en un hosting. Todas estas tareas ya figuran en el Backlog del tablero del proyecto.

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Anthropic. (2026). *Claude Code* (versión con el modelo Claude Opus 5.5) [Software]. https://claude.com/claude-code

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM, 13*(6), 377-387. https://doi.org/10.1145/362384.362685

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Halfond, W. G. J., Viegas, J., & Orso, A. (2006). A classification of SQL-injection attacks and countermeasures. *Proceedings of the IEEE International Symposium on Secure Software Engineering*, 13-15.

Härder, T., & Reuter, A. (1983). Principles of transaction-oriented database recovery. *ACM Computing Surveys, 15*(4), 287-317. https://doi.org/10.1145/289.291

Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3.ª ed.). Wiley.

Oracle Corporation. (s.f.). *MySQL 8.4 reference manual*. https://dev.mysql.com/doc/refman/8.4/en/

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

OWASP Foundation. (s.f.). *SQL injection prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

The PHP Group. (s.f.-a). PHP Data Objects. En *Manual de PHP*. https://www.php.net/manual/es/book.pdo.php

The PHP Group. (s.f.-b). Sentencias preparadas y procedimientos almacenados. En *Manual de PHP*. https://www.php.net/manual/es/pdo.prepared-statements.php

Anexos

Anexo A. Registro de uso de inteligencia artificial

Se utilizó Claude Code (Anthropic, 2026), con acceso al repositorio, en octubre de 2026. La IA intervino en el diseño del esquema, la implementación de los repositorios, las pruebas, la documentación, las capturas y la organización del tablero; el equipo definió el alcance, verificó cada resultado y asume la responsabilidad del contenido. La versión completa es docs/BITACORA_IA.md.

Tabla 9

Registro de uso de IA en la Actividad 2

#	Pedido del equipo	Propuesta de la IA	Verificación	Decisión
9	Modelo entidad-relación y script SQL	5 tablas InnoDB, 4 FK, UNIQUE y CHECK; 6 socios y 14 prendas	php bd/instalar.php; SUM(precio*stock) y conteo por socio recalculados a mano	Aceptada tras verificar
10	Conexión PDO fuera del código público	config/ fuera de public/ y de git; PDO con excepciones y sin emulación	Prueba de que la configuración no se descarga por URL; prueba de atributos	Aceptada
11	Repositorios con CRUD, validación y paginación	ProductoRepositorio con transacciones, SocioRepositorio, clase Pagina	22 pruebas de repositorios y CRUD web	Aceptada
12	Demostrar resistencia a la inyección SQL	13 ataques + revisión del código fuente + INYECCION_SQL.md	Todas en verde; tablas intactas tras cada ataque	Aceptada
13	Corregir pruebas que fallaban	Cuatro valores esperados incorrectos (Tabla 8)	Se investigó cada causa	Modificada
14	Migrar los datos de la Actividad 1	bd/migrar_json.php con transacción y --simular	Simulación: 7 omitidas y 1 nueva	Aceptada
15	Ejecutar en el Docker del curso y organizar el tablero	Contenedores MySQL 8.4 y PHP 8.3; tablero de sprint con 27 issues	56/57 en PHP 8.3: se halló la causa y se corrigió la prueba (57/57)	Modificada

Nota. Elaboración propia. La columna «Decisión» registra si la propuesta se aceptó tal cual o tuvo que modificarse después de verificarla. El equipo debe completar la reflexión en la bitácora.

Anexo B. Matriz de trazabilidad de la Actividad 2

Tabla 10

Matriz de trazabilidad: requisito, issue, implementación y prueba

Req.	Issue	Implementación	Prueba que lo verifica
RF-06	#13	SocioRepositorio, public/socios.php, socio_form.php, socio_eliminar.php	CRUD de socios (6 pruebas)
RF-07	#12, #13	paginar(), clase Pagina, componente paginacion()	Paginación de 8, 10 y 5; página 99 y -1
RF-08	#12	buscarPorCodigo() con JOIN a socios; public/producto.php	Ficha con su socio y 404
RF-09	#15	bd/migrar_json.php	Ejecución con --simular
RNF-06	#14	Consultas preparadas, SQL_ORDEN, addcslashes(), PARAM_INT	13 ataques + revisión del código
RNF-07	#11	config/config.ejemplo.php, .gitignore, src/config.php	Configuración protegida
RNF-08	#9, #10	bd/esquema.sql	FK inválida rechazada; UNIQUE; RESTRICT
RNF-09	#12	enTransaccion() en ProductoRepositorio	Transacción con ROLLBACK
RNF-10	#11	Manejador de errores en src/arranque.php	Base que no responde → 503 genérico

Nota. Elaboración propia. Todas las pruebas están en verificar.php; cobertura de trazabilidad: 9 de 9 requisitos (100 %).

Anexo C. Glosario

Clave foránea (FK): columna cuyo valor debe existir como clave primaria en otra tabla.

CRUD: operaciones de crear, leer, actualizar y eliminar (create, read, update, delete).

CSRF: falsificación de solicitudes entre sitios; se previene con un token secreto por sesión.

Inyección SQL: ataque que inserta código SQL en un dato para alterar la consulta.

PDO: PHP Data Objects, la capa de acceso a bases de datos de PHP.

PRG: patrón Post/Redirect/Get: tras un POST exitoso se redirige con 303 para evitar reenvíos.

Repositorio: objeto que concentra el acceso a los datos de una entidad.

Transacción: conjunto de operaciones que se aplican todas o ninguna.